

DefacementFusion: A Robust Multi-Modal Defacement Detection

Phan Hai Dang¹, Nguyen Trong Hung¹, Huynh Ngoc Khanh¹, Duc-Tho Mai^{2,*}

Abstract—Defacement attacks targeting websites have recently surged, becoming one of the most significant threats to organizations that offer services on web-based platforms. In response to this escalating threat, various defacement detection solutions have been proposed and implemented. However, many existing approaches face specific limitations: some are only effective for static websites or those with infrequently changing content, while others can support dynamic websites but require considerable computational resources or exhibit low detection accuracy due to inadequate analysis of the diverse components that make up a web interface, particularly the Hypertext Markup Language (HTML) structure. To tackle these issues, this paper introduces two main parts: HTMLDeface2vec, a model that pulls structural data from HTML to spot defacement attacks, and DefacementFusion, a deep learning model that detects issues by using three important features of a web page: its screenshot (image), text content (text), and HTML structure. DefacementFusion is engineered to function effectively on both static and dynamic web pages. The results from testing on a dataset of 96,220 web pages indicate that the system successfully detects attacks with an accuracy of 98.97% and a low false positive rate (FPR) of 1.24%, outperforming current detection methods.

Index Terms—Deep Learning, Website Defacement Detection, Multimodal Model, BERT, Web Application Security.

I. INTRODUCTION

Website defacement refers to an attack that changes the appearance of a website or web page by attacking the availability and integrity; however, it defaces the web page without access authorization [1], [2]. Defacement is a type of attack that targets websites, including changing the content, layout, or images of the site. The purpose of these attacks is often to alert the site's owner to patch security vulnerabilities; however, some hackers use website defacement to destroy the website, ruin the website owner's reputation, or cheat users by displaying fake information. Hackers often proceed with website defacement attacks to leave their signatures and messages on websites, similar to street art. Many hacker groups leave their signatures and messages on defaced sites to showcase their achievements.

Nonetheless, some groups perform defacements to warn site owners about their security flaws. In recent years, defacing websites and web applications has become a common tool to spread messages and viewpoints among cybercriminal groups. These attackers often target popular and important websites, including government websites, to spread misinformation,

promote distorted religious or political ideologies, or conduct other activities intended to affect the online community negatively. Due to the increasing prevalence of defacement attacks and their severe consequences on websites and web applications, many countermeasures have been researched, developed, and implemented in practice. Defacement detection methods are commonly categorized into anomaly-based, signature-based, and machine learning techniques. Among traditional anomaly-based methods, such as checksum comparison, DIFF tools, and signature-based detection, checksum comparison is the simplest and fastest. While these techniques are simple and provide rapid processing speed, they are only effective in monitoring static websites. In contrast, techniques such as statistical methods, genetic programming, and machine learning are more complex, requiring higher computational costs and slower processing times, but they can handle dynamic websites. Machine learning-based anomaly detection techniques have been instrumental in distinguishing defaced web pages from normal ones. The signature-based method stores known attack patterns (rules) to monitor webpages. When a match is found, it triggers an alarm for an attack. While fast and effective for known attack types, it cannot detect new or unknown attacks.

Burruss *et al.* [3] utilized a finite mixture model to classify website defacers into low-volume (69%) and high-volume (31%) groups based on attack volume across 119 active websites, providing insights into attacker behavior through factors like social media presence and defacement type, though limited to profiling rather than real-time detection. Conversely, Harahap *et al.* [4] developed a low-computational-cost method using predefined attack signatures to detect website defacement in university domains, leveraging patterns like keywords, Uniform Resource Locator (URL), or HTML structures, but its reliance on known signatures and lack of robust analysis for dynamic content or images restricts its effectiveness against novel attacks. Almomani *et al.* [5] introduced a machine learning model enhanced by Particle Swarm Optimization (PSO) for detecting URL defacement attacks (Enhanced URL Defacement Detection). This method optimizes feature selection to improve accuracy, achieving 92.5% on a test dataset with static content. However, the model faces challenges with dynamic websites due to its reliance on URL features and limited ability to process embedded multimedia content, such as images and videos, leading to an increased false detection rate when content changes frequently. Hoang *et al.* [6] developed a model combining supervised machine learning with signature-based detection and hash-

¹ Phan Hai Dang, Nguyen Trong Hung and Huynh Ngoc Khanh are with the Academy of People's Security, Hanoi 12511, Vietnam.

² Duc-Tho Mai is with the Academy of Cryptography Techniques (ACT), Hanoi 12511, Vietnam. Corresponding: duchtho-mai@actvn.edu.vn

based integrity checking for embedded files. This method is effective for static HTML content, but hash-based checking for multimedia content has limitations, as minor changes can evade detection, leading to a higher false detection rate. Wu *et al.* [7] utilized Support Vector Machine (SVM) for detecting defacement, extracting features from webpage content based on HTML. This method achieved an accuracy of 87.3% on the test dataset but faced challenges with a high false alarm rate, particularly when the detection threshold was not dynamically adjusted. It also lacked the ability to handle content with manually extracted features, leading to poor performance on websites with extensive images and diverse content. Hoang *et al.* [8] proposed a model that processes textual data of webpages to detect web defacement based on deep learning - BiLSTM, but this paper does not handle HTML structure or embedded content, limiting its effectiveness on complex websites with numerous images. In particular, Nguyen *et al.* [9] introduced a model that combines both textual data and image data from the screen. However, this combined model merely integrates the results of two independent models, each specialized for one data type. The final result is computed by averaging the evaluation scores of the two models. As a result, it fails to establish meaningful connections or mutual reinforcement between the text and the screenshot of the webpage.

Thus, it can be observed that the limitations of the existing website defacement detection solutions include (1) only considering URL-related content without focusing on processing webpage content, thus achieving low effectiveness in practice. (2) Effective only on static websites or static content. (3) High false alarm rate depending on the detection threshold. (4) Limited or partial handling of content, especially images and HTML structure. (5) Feature combination is limited to evaluation results for each data type. Based on the above analysis, this paper contributes the following key points:

- The paper introduces and develops a feature extraction framework based on a transformer model for HTML structure – HTMLDeface2vec. This framework facilitates the construction of a Document Object Model (DOM) tree and enables simultaneous encoding of both the semantic context of the HTML source code and the DOM tree structure of web pages
- This paper proposes DefacementFusion - a web defacement detection model that combines textual features, HTML structure features, and screenshots of web pages to improve detection accuracy and reduce false alarms.

The rest of this paper is organized as follows: Section II introduces the proposed approach, Section III presents and analyzes the experimental results. At last, in Section IV, we have a conclusion and address the future direction of this study.

II. PROPOSED APPROACH

This section introduces the proposed multi-modal model, *DefacementFusion*, which combines features from image, text, and HTML modalities. A key component of this model is

HTMLDeface2vec, a robust module designed to process the structure of HTML based on the BERT [10] architecture.

A. HTMLDeface2vec: Feature Extraction from HTML

When working with HTML data, the proposed method represents a webpage using its DOM tree. Each node in this tree contains both textual content and its corresponding HTML markup, while the overall tree structure captures the page layout and the relationships between different elements. However, websites that have undergone defacement attacks often exhibit disrupted layouts, resulting in fragmented and incomplete DOM trees. This makes it particularly challenging to encode the DOM as input for deep learning models for two main reasons: (1) Most existing text encoders are built to process linear, unstructured text and are not suited to handle the hierarchical nature of the DOM without customized pre-processing; (2) compared to typical sentences, even a flattened or serialized DOM tree tends to be excessively long—often comprising hundreds or thousands of nodes—which exceeds the input limits of many neural network architectures. Arbitrary segmentation of this sequence may lead to the loss of crucial contextual relationships.

1) *Preprocessing for the DOM Tree*: The objective is to segment a DOM tree into a collection of subtrees, ensuring that each one retains the essential contextual information of the tokens it contains. This process involves two main stages.

In the first stage, the DOM tree is cleaned by eliminating nodes that contribute minimally to the page's structure or semantics, such as `<script>` and `<style>` tags. At the same time, important descriptive attributes, like the values of `<class>` and `<id>` tags in HTML elements, are preserved, as they provide valuable context for understanding the role and identity of each element within the page. However, even after the cleaning stage, many DOM trees may still contain thousands of data points and tokens, many of which carry limited semantic value. This results in excessive input lengths that are difficult to handle with most current deep learning models, making the direct processing of the entire DOM inefficient and impractical. Next, the DOM tree is segmented into overlapping subtrees using a fixed token size sliding window m . Unlike traditional text-based sliding windows, which risk breaking important parent-child relationships, this approach ensures that semantically relevant nodes remain together. Starting from the root, the tree is traversed in pre-order to construct the first subtree containing up to m tokens. Subsequent subtrees are generated by incrementally expanding the current subtree with new nodes and pruning distant ones, preserving meaningful context throughout the tree. For example, as shown in Figure 1, illustrating pre-order processing with a maximum of 5 tokens. At each step in the process, new nodes are added to the subtree, while older nodes beyond the current window are removed. The resulting subtree at the end of the process includes all the nodes that were most recently added and those retained from previous steps.

2) *DOM Encoder*: As previously mentioned, the goal is to enhance existing language model architectures, such as BERT

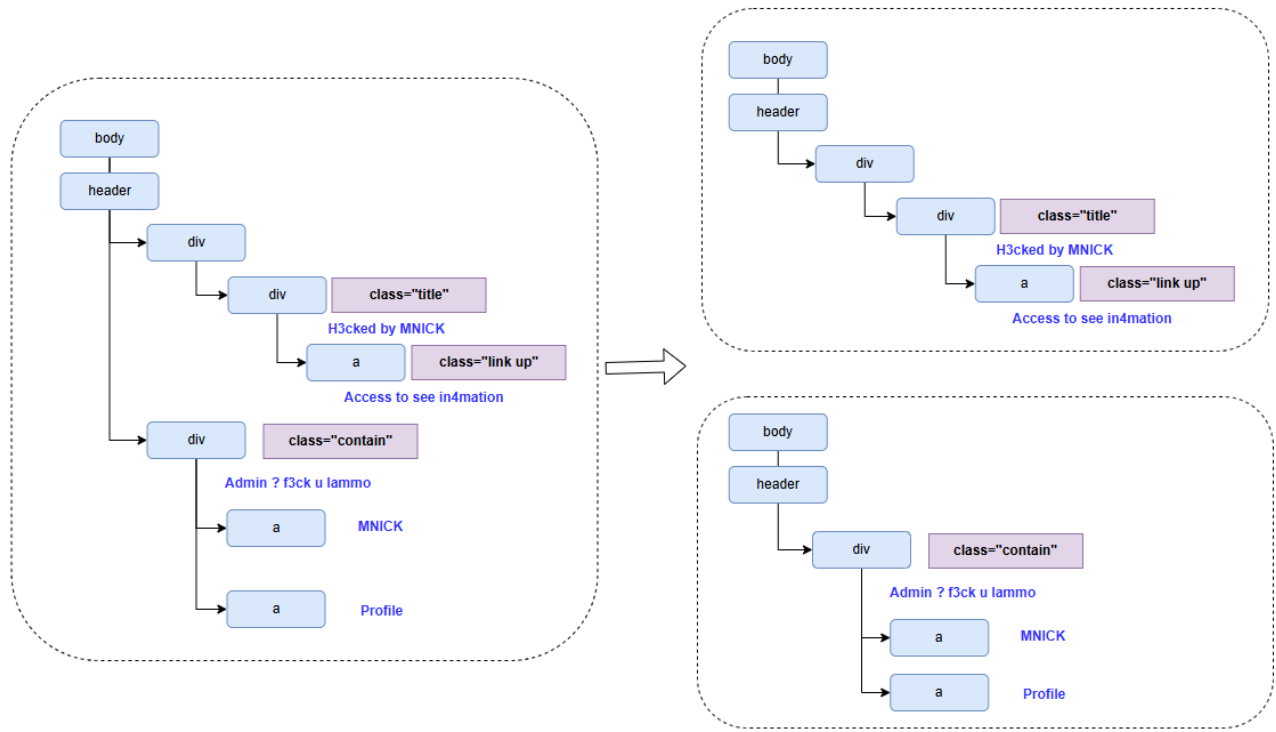


Fig. 1: DOM Tree Processing with Subtrees (Pre-order and Max 5 tokens).

[10], to effectively capture both structural and textual semantics of the DOM. To this end, the transformer encoder from BERT is utilized as the foundational encoding component. At the heart of this encoder lies the self-attention mechanism [11], which enables the model to generate contextualized representations of each token by dynamically aggregating information from all other tokens in the input sequence. This mechanism allows the model to capture both local and global dependencies, making it well-suited for representing the hierarchical and semantic richness of HTML content.

There are two fundamental distinctions between unstructured text and the DOM tree context that call for the development of specialized positional embeddings suited to tree-structured data: (1) In contrast to linear text, tokens in a DOM tree are organized hierarchically rather than sequentially. This means that, beyond capturing intra-node token relationships, it is also essential to incorporate contextual information from tokens residing in other nodes throughout the tree. (2) The DOM structure allows for various notions of relative positioning between tokens or nodes, necessitating the use of differentiated positional embeddings to effectively represent these diverse spatial relationships within the tree.

3) *Pretrain model*: Given the transformer-based architecture of the DOM structure encoder, it is possible to leverage parameters from pre-trained language models such as BERT [10]. However, since BERT is trained exclusively on unstructured textual data, it may effectively model the textual semantics within individual DOM nodes but lacks the capacity to capture the structural and layout characteristics inherent in

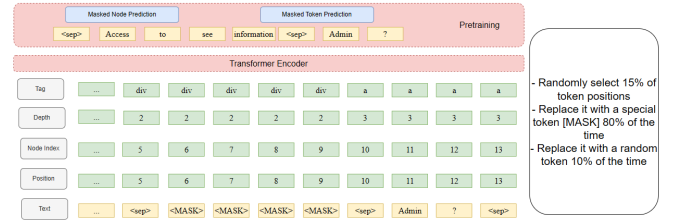


Fig. 2: Pretrain model.

HTML DOM trees. To bridge this gap, we further pre-train the model on a large collection of unlabeled web pages using the Masked Language Modeling objective adopted from BERT.

In particular, the approach involves initially selecting 15% of token positions at random. DOM nodes are then sampled iteratively, and all tokens within each sampled node are added to the mask set until the total number of masked tokens reaches the 15% threshold. Following BERT's masking strategy, for each selected token position: (1) 80% of the time it is replaced with the special token [MASK]; (2) 10% of the time it is replaced with a randomly chosen token; and (3) in the remaining 10%, the token is left unchanged, as illustrated in Figure 2.

B. Feature extraction from text and image data

For text and image data, the proposed model uses methods similar to Nguyen *et al.* [9]: For the text data extracted from web pages, pre-processing is performed to extract features for

training, forming a vector representation for each page using a Tokenizer. For the screenshot image data, preprocessing is done in the following steps: standardized to the same size 224×224 . The value of each pixel in the screen capture image is normalized to the range $[0, 1]$.

C. Multi-modal defacement detection

The proposed model utilizes a multimodal feature fusion approach that combines features extracted from the HTML source code, text, and screenshots of web pages. The advantage of this approach is that it allows the model to combine features extracted from the HTML source code, text data, and screenshots of web pages. HTMLDeface2vec is an assumed feature extraction model designed to encode the structure and content of the DOM HTML tree. The DOM tree is divided into subtrees using a DOM tree processor algorithm. Each subtree is encoded into a representation vector through HTMLDeface2vec, capturing both the textual content of the nodes and the structural relationships between them (e.g., parent-child, sibling). The output is a set of feature vectors H . The textual content of the web page is extracted and processed as a tokenized sequence. BiLSTM encodes the token sequence in both directions, creating rich contextual representations. This results in a set of feature vectors T . Screenshots related to the web page are collected. EfficientNetB0 [12] processes each image to generate feature vectors that capture visual elements such as layout, colors, and objects. The output of EfficientNet is a set of feature vectors of images I .

The feature vectors of the three extraction models (HTML, Text, Image) are combined into a unified set $F = H \cup T \cup I$. The set F is then fed into a transformer encoder with self-attention. This process allows the model to compute the relationships between the feature vectors, regardless of their origin (HTML, text, or image). Finally, the model passes the extracted features from the transformer into a multilayer perceptron (MLP) to classify whether the input web page has been defaced or is normal.

Figure 3 illustrates the overall architecture of the proposed multimodal DefacementFusion model, which consists of three main stages: pretraining, classification training, and classification testing. In the pretraining stage, HTML documents are processed through DOM tree preprocessing and fed into a BERT-base-uncased model to generate pretrained representations. The classification training stage combines three parallel processing streams: HTML features through a pretrained model, text features via BiLSTM, and image features using EfficientNetB0. These multi-modal features are then fused through a Transformer layer and classified using an MLP to distinguish between defaced and benign web pages.

III. EXPERIMENTS

A. Datasets

Previous studies in this domain have shown several limitations, such as using statistical methods without proper datasets, relying on limited or confidential data, and focusing only on specific modalities, like text or images, thus reducing detection

effectiveness. To ensure a fair evaluation, we re-implemented representative methods and developed a custom dataset for training and testing. The dataset includes both normal and defaced websites: normal data were collected from Alexa-ranked sites [13], while defaced samples were obtained from Zone-h.org [14]. Using a toolset based on Selenium WebDriver [15] and BeautifulSoup, HTML, text, and page screenshots are extracted from the webpage. In total, the dataset comprises 57,120 labeled normal samples and 39,100 defaced samples.

B. Implementation Details

In the pre-training phase, unlabeled HTML data from a custom-built dataset was utilized. The encoder was based on the BERT-base-uncased architecture, comprising 110 million parameters and a hidden size of 768. It was initialized with publicly available BERT-base-uncased weights and further pre-trained on the HTML dataset for 5 epochs, using a batch size of 16 and a learning rate of $2e^{-5}$. After pre-training and fine-tuning, the encoder was employed in both experimental settings. For classification tasks, a two-layer feedforward neural network was attached to the encoder's output: the first layer includes 256 hidden units with ReLU activation, followed by an output layer producing logits for two classes—defaced and benign. This architecture enabled effective discrimination between normal and defaced web pages.

All training and inference procedures in the classification stages were conducted using the system configuration as shown in Table I.

TABLE I: System Configuration

Component	Specification
CPU	CPU Intel Xeon (2.6 GHz)
GPU	NVIDIA Tesla P100 (VRAM 16GB)
RAM	29 GB
Epochs	10
Batch Size	16
Optimizer	Adam
Learning Rate	$2e^{-5}$
Loss function	Binary Cross Entropy

C. Results

Using the constructed dataset with an 80% training and 20% testing split, experiments on various proposed defacement detection models, including those by Random Forest [6], BiLSTM [8], and EfficientNet; BiLSTM; BiLSTM + EfficientNet [9]. We also evaluated the approach by Wu *et al.* [7], as well as the models proposed in this study, which include processing HTML data using a pretrained BERT-Base encoder, the model utilizing HTMLDeface2vec, and the feature-fusion model DefacementFusion.

Figure 4 shows the training and validation accuracy curves for HTMLDeface2vec during the training process. Both curves exhibit rapid improvement in the initial epochs, with the most significant gains occurring early in training. The small gap between training and validation curves throughout the process indicates excellent generalization with minimal overfitting.

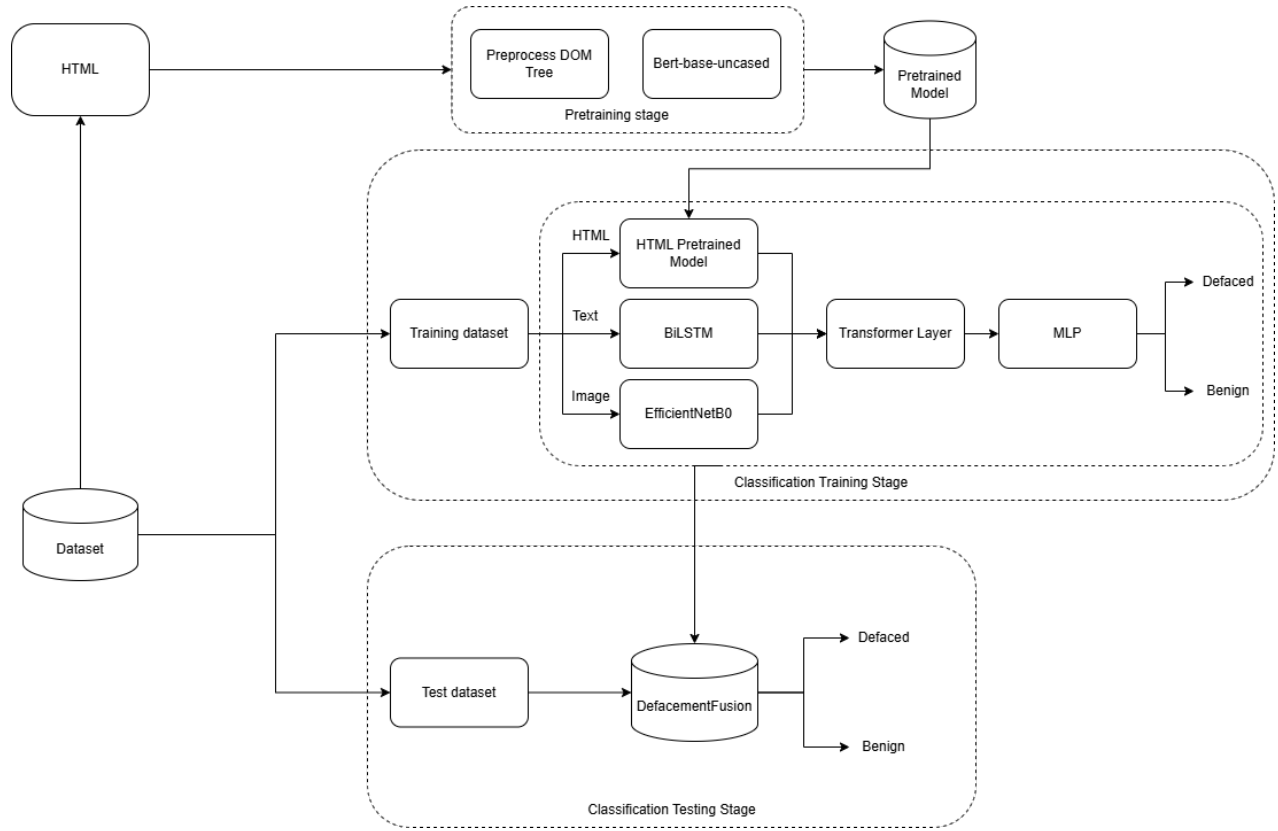


Fig. 3: The Proposed Multi-modal Model

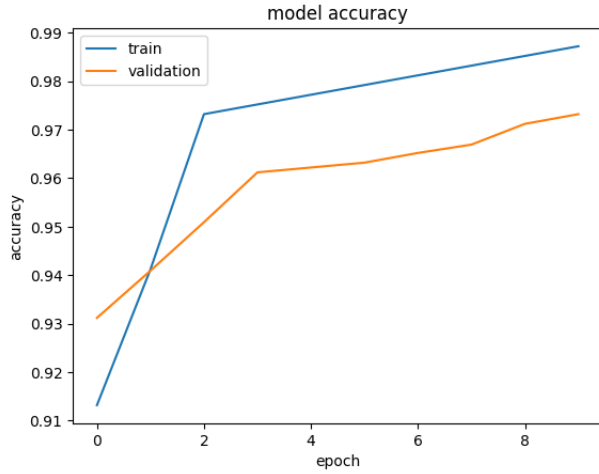


Fig. 4: Training and validation accuracy over epochs during the model training process of HTMLDeface2vec only.

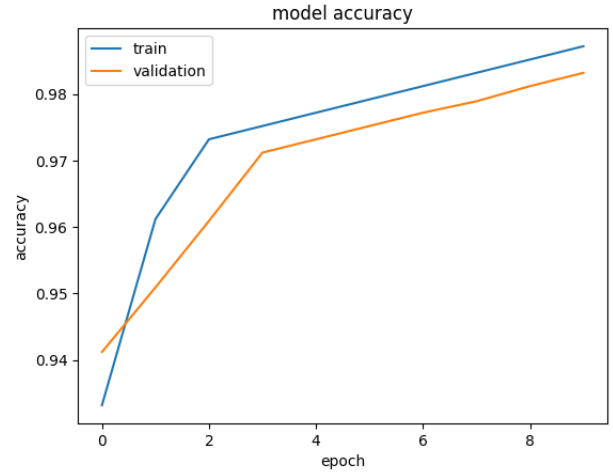


Fig. 5: Training and validation accuracy over epochs during the model training process of DefacementFusion.

The steady convergence demonstrates the effectiveness of the DOM tree preprocessing approach.

Figure. 5 presents similar accuracy trends, confirming the robustness of our model. The curves show consistent behavior with rapid early convergence followed by gradual improvement. The close alignment between training and validation performance further validates the generalization capability and

accurate detection potential of *DefacementFusion* approach.

Table II presents the experimental results of models using HTML features to address *HTMLDeface2vec* features, which are effective for detecting defacement attacks. In this context, BERT-Base refers to the baseline model employing the pretrained BERT-Base encoder, while HTMLDeface2vec denotes the model after applying the proposed feature extrac-

TABLE II: Performance Comparison of Models Using HTML Feature

Model	Accuracy	Precision	Recall	F1-score	FPR
Hoang et al [6]	86.03	94.28	69.35	79.92	2.88
Wu et al. [7]	91.28	91.26	91.28	91.26	5.98
BERT-base-uncased	90.56	93.50	82.63	87.73	3.93
HTMLDeface2vec (Ours)	95.79	94.81	93.59	94.20	3.51

tion method. The proposed *HTMLDeface2vec* model achieves significantly better results on the dataset, reaching an accuracy of **95.79%**, which is over **4%** higher than the best-performing previous model by Wu *et al.* [7]. In addition to accuracy, our model also demonstrates superior performance in terms of F1-score. This highlights the strength of *HTMLDeface2vec* in effectively detecting web defacement attacks with a notably better balance between detection accuracy and FPR compared to both previous models and the baseline BERT-based approach.

TABLE III: Performance Comparison of Various Models

Model	Accuracy	Precision	Recall	F1-score	FPR
Hoang et al [6]	86.03	94.28	69.35	79.92	2.88
Wu et al [7]	91.28	91.26	91.28	91.26	5.98
Hoang et al [8]	96.54	96.93	94.43	95.66	2.05
Nguyen et al [9]	97.49	97.76	95.99	96.87	1.51
BERT-base-uncased	90.56	93.50	82.63	87.73	3.93
DefacementFusion (Ours)	98.97	98.76	98.39	98.57	1.24

Table III shows the experimental results of various defacement detection models to address *DefacementFusion* effective for detecting defacement attacks. The *DefacementFusion* model is a feature fusion approach that combines HTML data processed by the proposed HTMLDeface2vec method, textual data processed by the BiLSTM model, and visual data processed by the EfficientNetB0 model. DefacementFusion demonstrates outstanding performance in detecting web defacement attacks, significantly surpassing previously proposed methods. The proposed model reaches an accuracy of 98.97% and an F1-score of 98.57%, while maintaining a FPR as low as 1.24%. These results clearly indicate the superior effectiveness of *DefacementFusion* in identifying defacement attacks compared to existing models.

IV. CONCLUSION

This research has proposed a novel method for detecting web defacement attacks based on a multimodal deep learning approach. By integrating three distinct feature streams - screenshots, textual content, and HTML structure - the DefacementFusion model can identify anomalies across multiple webpage aspects. A significant contribution of this research is developing the HTMLDeface2vec framework, which efficiently extracts feature vectors from HTML structures - a data source often overlooked in previous methods. This process automates source code analysis and transforms it into vector representations, enabling machine learning models to detect patterns characteristic of defacement attacks. Experimental results on a large dataset of 96,220 webpages have confirmed the superior effectiveness of the proposed method, achieving 98.97% accuracy and a false positive rate of only 1.24%.

Notably, the model demonstrates good detection capability on static and dynamic webpages, addressing a significant limitation of traditional methods. This detection method improves accuracy while maintaining acceptable computational performance, though it still requires considerable resources, which may limit its scalability in some real-world web security deployments.

REFERENCES

- [1] M. Albalawi, R. Aloufi, N. Alamrani, N. Albalawi, A. Aljaedi and A.R. Alharbi, "Website defacement detection and monitoring methods: A review," *Electronics*, vol. 11, no. 21, p. 3573, 2022, doi:10.3390/electronics11213573
- [2] Tanvi. and Dr. A. Bhandari, "ML-Based Methods for Detecting Phishing Websites: A Comprehensive Survey and Analysis," *SSRN Journal*, 2024, doi:10.2139/ssrn.4829465.
- [3] G. W. Burruss, C. J. Howell, D. Maimon, and F. Wang, "Website defacer classification: A finite mixture model approach," *Social Science Computer Review*, vol. 40, no. 3, pp. 775–787, 2022, doi:10.1177/0894439321994232.
- [4] H. Harahap and F. Ridho, "Detection of online gambling web defacement in university domains using attack signatures," in *Proc. 2024 Int. Conf. on Artificial Intelligence, Blockchain, Cloud Computing, and Data Analytics (ICoABCD)*, pp.73–78, 2024, doi:10.1109/ICoABCD63526.2024.10704413
- [5] O. Almomani, A. Alsaaidah, A. A. Abu-Shareha, A. Alzaqebah, M. A. Almaiah, and Q. Shambour, "Enhance URL Defacement Attack Detection Using Particle Swarm Optimization and Machine Learning," *Journal of Computational and Cognitive Engineering*, Feb.14,2025, doi:10.47852/bonviewJCCE52024668.
- [6] X. D. Hoang and N. T. Nguyen, "Detecting Website Defacements Based on Machine Learning Techniques and Attack Signatures," *Computers*, vol. 8, no. 2, p. 35, 2019, doi:10.3390/computers8020035.
- [7] S. Wu, X. Tong, W. Wang, G. Xin, B. Wang, and Q. Zhou, "Website defacements detection based on support vector machine classification method," in *Proc. 2018 Int. Conf. on Computing and Data Engineering (ICCDE)*, Shanghai, China, May 4–6, 2018, pp.62–66, doi:10.1145/3219788.3219804.
- [8] X. D. Hoang, T. H. Nguyen, and H. D. Pham, "A novel model for detecting web defacement attacks transformer using plain text features," *Indonesian Journal of Electrical Engineering and Computer Science*, vol.37, no.1, pp.232–240, Jan.2025, doi:10.11591/ijeecs.v37.i1.pp232-240.
- [9] T. H. Nguyen, X. D. Hoang, and D. D. Nguyen, "Detecting website defacement attacks using web-page text and image features," *Int. J. Adv. Comput. Sci. Appl.*, vol.12, no.7, pp.215–222, Aug.2021, doi:10.14569/IJACSA.2021.0120725.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, vol.1 (Long and Short Papers), Minneapolis, MN, USA, Jun. 2–7, 2019, pp.4171–4186, doi:10.18653/v1/N19-1423.
- [11] A.Vaswani, N.Shazeer, N.Parmar, J.Uszkoreit, L.Jones, A.N.Gomez, Ł.Kaiser, and I.Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, Long Beach, CA, USA, Dec. 2017.
- [12] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. 36th Int. Conf. on Machine Learning (ICML 2019)*, vol.97, P.Chaudhuri and R. Salakhutdinov, Eds., Jun.2019, pp.6105–6114.
- [13] "Alexa, Top 1 million domains." [Online]. Available: <https://www.kaggle.com/datasets/cheedcheed/top1m?select=top-1m.csv> (accessed on June 2025).
- [14] "Zone-H.org." [Online]. Available: <http://zone-h.org/?hz=1> (accessed June 2025).
- [15] "Selenium." [Online]. Available: <https://www.selenium.dev/> (accessed on June 2025).